Large Language Models & Generative AI

How LLMs work, how to use, adapt, evaluate and build applications and agents with them.

Contents

#	Topic	Level
1	What are LLMs? Tokens, context window, parameters, scaling laws	Beginner
2	LLM training pipeline: pre-training, SFT, RLHF/DPO	Intermediate
3	Calling LLM APIs (Anthropic, OpenAI, Gemini, open-source via Ollama / vLLM)	Beginner
4	Prompt engineering	Beginner
5	Structured output, function/tool calling	Intermediate
6	Embeddings and vector search	Intermediate
7	Retrieval-Augmented Generation (RAG)	Intermediate
8	Chunking, re-ranking, hybrid search, evaluating RAG	Advanced
9	Fine-tuning: full, LoRA, QLoRA, PEFT	Advanced
10	Fine-tuning vs RAG vs prompting - when to use which	Intermediate
11	Agents: ReAct, tool use, planning, memory, multi-agent	Advanced
12	Model Context Protocol (MCP) and tool ecosystems	Advanced
13	LLM evaluation and benchmarking	Advanced
14	Hallucinations, safety, jailbreaks, prompt injection, guardrails	Advanced
15	Running open-source LLMs locally: Llama, Mistral, Qwen, Gemma; Ollama, llama.cpp, vLLM	Intermediate
16	Inference optimisation: KV cache, batching, speculative decoding, quantisation	Advanced
17	Multimodal models: vision, audio, video, documents	Advanced
18	Image/video/audio generation: Stable Diffusion, Flux, Sora-style video, TTS, music	Advanced
19	Build projects: chat with PDF, SQL agent, code assistant, customer-support bot	Intermediate
20	Generative AI ethics, copyright, licensing and responsible AI	Intermediate

1. What are LLMs? Tokens, context window, parameters, scaling laws [Beginner]

Explanation (English)

LLMs are large Transformers trained to predict the next token on huge text corpora. Capability grows with parameters, data and compute. Context window limits how much text they see.

Tanglish (Tamil + English) Vilakkam

LLM = periya Transformer, next token predict pannum. Context window = ethanai tokens paarkum.

Learn more

YouTube search: intro to large language models karpathy

2. LLM training pipeline: pre-training, SFT, RLHF/DPO [Intermediate]

Explanation (English)

Pre-train on web-scale text; supervised fine-tune (SFT) on instruction data; align with human feedback (RLHF, DPO, RLAIF/constitutional methods).

Tanglish (Tamil + English) Vilakkam

Pretraining -> SFT -> RLHF/DPO. Apdi dhaan ChatGPT/Claude create aagudhu.

Learn more

YouTube search: rlhf dpo explained

3. Calling LLM APIs (Anthropic, OpenAI, Gemini, open-source via Ollama / vLLM) [Beginner]

Explanation (English)

Send messages with a system prompt and parameters (temperature, max_tokens, top_p); stream responses; handle rate limits and costs.

Tanglish (Tamil + English) Vilakkam

API-la prompt anuppi response vaanga. temperature, max_tokens.

Example / Program

```
import anthropic
c=anthropic.Anthropic()
r=c.messages.create(model='claude-sonnet-5-5',max_tokens=200,system='Reply in Tanglish',messages=[{'role':'user','content
print(r.content[0].text)
```

Learn more

YouTube search: Calling LLM APIs (Anthropic, OpenAI, Gemini, open-source via Ollama / vLLM) tutorial

4. Prompt engineering [Beginner]

Explanation (English)

Be clear and specific, give role and context, few-shot examples, ask for step-by-step reasoning or structured output (JSON), iterate and test prompts on an eval set.

Tanglish (Tamil + English) Vilakkam

Prompt clear-a, examples-oda, output format sollunga. Test pannunga.

Example / Program

```
prompt='''You are a data scientist.
Task: classify sentiment as positive/negative.
Return JSON {"label":...}
Text: I loved it'''
```

Learn more

YouTube search: Prompt engineering tutorial

5. Structured output, function/tool calling [Intermediate]

Explanation (English)

Make models return JSON schemas or call functions/tools you define; essential for reliable apps and agents.

Tanglish (Tamil + English) Vilakkam

LLM JSON return pannum, tools call pannum.

Example / Program

```
tools=[{'name':'get_weather','description':'Get weather','input_schema':{'type':'object','properties':{'city':{'type':'st
```

Learn more

YouTube search: Structured output, function/tool calling tutorial

6. Embeddings and vector search [Intermediate]

Explanation (English)

Convert text to vectors; store in a vector DB; retrieve nearest chunks by cosine similarity (ANN: HNSW, IVF).

Tanglish (Tamil + English) Vilakkam

Embeddings + vector DB = semantic retrieval.

Example / Program

```
from sentence_transformers import SentenceTransformer
import faiss,numpy as np
m=SentenceTransformer('all-MiniLM-L6-v2');d=['ml','cooking','ai'];e=m.encode(d).astype('f')
i=faiss.IndexFlatIP(e.shape[1]);faiss.normalize_L2(e);i.add(e)
q=m.encode(['deep learning']).astype('f');faiss.normalize_L2(q);print(d[i.search(q,1)[1][0][0]])
```

Learn more

YouTube search: Embeddings and vector search tutorial

7. Retrieval-Augmented Generation (RAG) [Intermediate]

Explanation (English)

Load documents -> chunk -> embed -> store -> retrieve top-k for a question -> put into prompt -> generate grounded answer with citations. Improve with hybrid search, re-ranking, query rewriting, metadata filters.

Tanglish (Tamil + English) Vilakkam

RAG: unga documents-la search panni, LLM-ku context-a kudukkum. Hallucination kuraiyum.

Example / Program

```
def rag(q,store,llm,k=3):
    ctx='\n'.join(store.search(q,k))
    return llm(f'Answer using only this context:\n{ctx}\n\nQ:{q}')
```

Learn more

YouTube search: Retrieval-Augmented Generation (RAG) tutorial

8. Chunking, re-ranking, hybrid search, evaluating RAG [Advanced]

Explanation (English)

Chunk size/overlap by structure; BM25 + dense hybrid; cross-encoder re-rankers; evaluate retrieval (recall@k, MRR) and answers (faithfulness, relevance - RAGAS).

Tanglish (Tamil + English) Vilakkam

Chunking, reranking, hybrid search - RAG quality improve.

Learn more

YouTube search: advanced rag techniques reranking hybrid search

9. Fine-tuning: full, LoRA, QLoRA, PEFT [Advanced]

Explanation (English)

Adapt a base model with your data. LoRA trains small low-rank adapter matrices; QLoRA does so on a 4-bit quantised base so it fits on a single GPU.

Tanglish (Tamil + English) Vilakkam

LoRA konjam parameters mattum train. QLoRA 4-bit - small GPU-la kooda possible.

Example / Program

```
from peft import LoraConfig,get_peft_model
cfg=LoraConfig(r=8,lora_alpha=16,target_modules=['q_proj','v_proj'],task_type='CAUSAL_LM')
# model=get_peft_model(base_model,cfg);model.print_trainable_parameters()
```

Learn more

YouTube search: Fine-tuning: full, LoRA, QLoRA, PEFT tutorial

10. Fine-tuning vs RAG vs prompting - when to use which [Intermediate]

Explanation (English)

Prompting first, RAG for fresh/private knowledge, fine-tuning for style, format, behaviour or latency/cost savings. Often combine.

Tanglish (Tamil + English) Vilakkam

Prompt -> RAG -> Fine-tune order-la try pannunga.

Learn more

YouTube search: rag vs fine tuning vs prompt engineering

11. Agents: ReAct, tool use, planning, memory, multi-agent [Advanced]

Explanation (English)

An agent loops: think -> choose a tool -> observe result -> continue until done. Add planning, memory, guardrails and human-in-the-loop. Frameworks: LangGraph, Claude Agent SDK, CrewAI, AutoGen.

Tanglish (Tamil + English) Vilakkam

Agent = LLM + tools + loop. Self-a decide panni tools use pannum.

Example / Program

```
while not done:
    action=llm(history)          # decide
    result=tools[action.name](**action.args)
    history.append(result)
```

Learn more

YouTube search: Agents: ReAct, tool use, planning, memory, multi-agent tutorial

12. Model Context Protocol (MCP) and tool ecosystems [Advanced]

Explanation (English)

An open protocol to connect LLM apps to tools and data sources through standard servers.

Tanglish (Tamil + English) Vilakkam

MCP = LLM-ku tools & data connect panna standard protocol.

Learn more

YouTube search: model context protocol mcp tutorial
MCP site: https://modelcontextprotocol.io

13. LLM evaluation and benchmarking [Advanced]

Explanation (English)

Build task-specific eval sets; use exact match, rubric/LLM-as-judge, human review; public benchmarks (MMLU, GSM8K, HumanEval, SWE-bench); track regressions.

Tanglish (Tamil + English) Vilakkam

Eval set create pannunga. LLM-as-judge, human review.

Learn more

YouTube search: llm evaluation llm as a judge

14. Hallucinations, safety, jailbreaks, prompt injection, guardrails [Advanced]

Explanation (English)
Ground answers, validate outputs, sanitise untrusted content, least-privilege tools, red-team, moderation filters, privacy/PII controls.

Tanglish (Tamil + English) Vilakkam
Hallucination kuraikka grounding. Prompt injection-ku careful.

Learn more
YouTube search: prompt injection llm security

15. Running open-source LLMs locally: Llama, Mistral, Qwen, Gemma; Ollama, llama.cpp, vLLM [Intermediate]

Explanation (English)
Download weights from Hugging Face, quantise (GGUF/AWQ/GPTQ), serve with Ollama / vLLM / TGI. Pick by licence, size and hardware.

Tanglish (Tamil + English) Vilakkam
Local-la LLM run: Ollama, llama.cpp. Quantization memory kuraikkum.

Example / Program

```
# ollama pull llama3.2 && ollama run llama3.2
import requests
print(requests.post('http://localhost:11434/api/generate',json={'model':'llama3.2','prompt':'hi','stream':False}).json()[
```

Learn more
YouTube search: Running open-source LLMs locally: Llama, Mistral, Qwen, Gemma; Ollama, llama.cpp, vLLM tutorial

16. Inference optimisation: KV cache, batching, speculative decoding, quantisation [Advanced]

Explanation (English)
KV cache avoids recomputation; continuous batching raises throughput; speculative decoding speeds generation; FlashAttention reduces memory.

Tanglish (Tamil + English) Vilakkam
Inference fast-a pannum techniques.

Learn more
YouTube search: llm inference optimization kv cache flash attention

17. Multimodal models: vision, audio, video, documents [Advanced]

Explanation (English)
Models that accept images/audio/PDF and text; build apps like chart readers, OCR assistants and voice agents.

Tanglish (Tamil + English) Vilakkam
Multimodal = image, audio, text serthu.

Example / Program

```
# send an image block + text to a vision-capable chat model
```

Learn more
YouTube search: Multimodal models: vision, audio, video, documents tutorial

18. Image/video/audio generation: Stable Diffusion, Flux, Sora-style video, TTS, music [Advanced]

Explanation (English)
Diffusion/flow models with text conditioning; LoRA for styles; ControlNet for structure; evaluate with human preference and CLIP score.

Tanglish (Tamil + English) Vilakkam
Image/video/music generate panna diffusion models.

Learn more
YouTube search: stable diffusion lora controlnet tutorial

19. Build projects: chat with PDF, SQL agent, code assistant, customer-support bot [Intermediate]

Explanation (English)
Combine API + RAG + tools + UI (Streamlit/Gradio/Next.js) + eval + deployment. Log traces (LangSmith/Langfuse).

Tanglish (Tamil + English) Vilakkam
PDF chatbot, SQL agent, code assistant projects.

Learn more
YouTube search: build rag chatbot python pdf

20. Generative AI ethics, copyright, licensing and responsible AI [Intermediate]

Explanation (English)
Bias, fairness, privacy, transparency, copyright and dataset licences; model cards; regulation (EU AI Act); energy use.

Tanglish (Tamil + English) Vilakkam
Bias, privacy, copyright, licence - responsible AI.

Learn more
YouTube search: responsible ai ethics generative ai